Comments in python:

- Including comments in programs makes code more readable for humans as it provides some information or explanation about what each part of a program is doing.
- Depending on the purpose of your program, comments can serve as notes to yourself or reminders, or they can be written with the intention of other programmers being able to understand what your code is doing.
- In general, it is a good idea to write comments while you are writing or updating a program as it is easy to forget your thought process later on, and comments written later may be less useful in the long term.
- Comments in Python begin with a hash mark (#) and whitespace character and continue to the end of the line.

```
# This is a comment
```

- In a “Hello, World!” program, a comment may look like this:

```
# Print “Hello, World!” to console
print("Hello, World!")
```

- Inline comments occur on the same line of a statement, following the code itself. Like other comments, they begin with a hash mark and a single whitespace character.
- Generally, inline comments look like this:

```
[code]  # Inline comment about the code
```

- Example:

```
z = 2.5 + 3j  # Create a complex number
```

- Multiline comments in python can be placed in between triple single quotes **‘’’** or triple double quotes **“”””**
- Example:

```
'''following program performs
Addition of two integer numbers
And prints the result on console'''
a=10
b=20
c=a+b
print(c)
```

Data types in python:

Python has following standard Data Types:

data type	Description	Nature	Example
int	Integer, is a whole number, positive or negative, without decimals, of unlimited length.	Immutable (un-changeable)	i=25 j=234456778800 type(i)#<class 'int'> type(j)#<class 'int'> print(i) #25 print(j) # 234456778800
float	"Floating point number" is a number, positive or negative, containing one or more decimals.	Immutable	f=23.456 type(f) #<class 'float'> print(f)
complex	Complex numbers Form: x+yj x: real part y: imaginary part	Immutable	c1=20+50.25j type(c1) #<class 'complex'> print(c1) #20+50.25j print(c1.real)# 20 print(c1.imag) # 50.25
str	Strings: combination of characters. Each character in string can be accessed by index. Positive index starts from '0'(left to right). Negative index starts from'-1'(right to left).	Immutable	s1='hello' s2="HELLO" s3='''hello python''' type(s1)# <class 'str'> print(s2) # HELLO print(s3[2])# l
bool	Boolean: True or False	Immutable	b1=True b2=False type(b1)#<class 'bool'> print(b2)# False

list	List is a sequence of ordered homogeneous elements. Symbol: [ ]
	Supports indexing
	(positive,
	negative).
		Mutable	`lst=[1,2,3,4,5]`
			`type(lst)# <class 'list'>`
			`print(lst)# [1,2,3,4,5]`
			`print(lst[2])# 3`
			`print(lst[-2])# 4`

tuple	tuple is a sequence of ordered heterogeneous objects.
	Symbol: ( )
	Supports indexing (positive, negative).
		Immutable	`tup=(10,'hello',True)`
			`type(tup)# <class 'tuple'>`
			`print(tup[0]) #10`
			`print(tup[-2]) #hello`

set	Set is a collection of un- ordered elements.
	Symbol: { }
	Discards the duplicates.
		Mutable	`vowels={'a','e','i','o','u'}`
			`type(vowels)# <class 'set'>`
			`print(vowels)`
			`# {'a','e','i','o','u'}`

dict	Dictionaries are used for mapping (key,value) pairs.	Mutable	`d={1:'a',2:'b',3:'c'}`
			`type(d) # <class 'dict'>`
			`print(d) #{1:'a',2:'b',3:'c'}`

****

Type Conversion functions:

- Python defines type conversion functions to directly convert one data type to another which is useful in day to day and competitive programming.
- Following are some conversion functions.

int(a, base) : This function converts any data type to integer. 'Base' specifies the base in which string is if data type is string.
float() : This function is used to convert any data type to a floating point number

Program:

.

```
s ="10010"
c =int(s,2)
print ("After converting to integer base 2 :",c)
e =float(s)
print ("After converting to float :",e)
```

Output:

```
After converting to integer base 2 : 18

After converting to float : 10010.0
```

ord() : This function is used to convert a character to integer. **hex() :** This function is to convert integer to hexadecimal string. **oct() :** This function is to convert

integer to octal string.

Program:

```
s = '4'
c = ord(s)
print ("After converting character to integer :",c)
print (c)
d = hex(56)
print ("After converting 56 to hexadecimal string :
",d)
e = oct(56)
print ("After converting 56 to octal string : ",e)
```

Output:

```
After converting character to integer : 52
After converting 56 to hexadecimal string : 0x38
After converting 56 to octal string : 0o70
```

tuple() : This function is used to convert to a tuple.
set() : This function returns the type after converting to set.
list() : This function is used to convert any data type to a list type.

```
s = 'geeks'
t = tuple(s)
print ("After converting string to tuple :",t)
st = set(s)
print ("After converting string to set :",st)
l = list(s)
print ("After converting string to list :",l)
```

Output:

```
After converting string to tuple : ('g', 'e', 'e', 'k', 's')
After converting string to set : {'k', 'e', 's', 'g'}
After converting string to list : ['g', 'e', 'e', 'k', 's']
```

dict() : This function is used to **convert a tuple of order (key,value) into a dictionary**.
str() : Used to **convert integer into a string.**
complex(real,imag) : : This function **converts real numbers to complex(real,imag) number.**

```
# Python code to demonstrate Type conversion
# using  dict(), complex(), str()

# initializing integers
a = 1
b = 2

# initializing tuple
tup = (('a', 1) ,('f', 2), ('g', 3))
# printing integer converting to complex number
c = complex(1,2)
print ("After converting integer to complex number : ",end="")
print (c)
```

```
# printing integer converting to string
c = str(a)
print ("After converting integer to string : ",end="")
print (c)

# printing tuple converting to expression dictionary
c = dict(tup)
print ("After converting tuple to dictionary : ",end="")
print (c)
```

Output:

```
After converting integer to complex number : (1+2j)
After converting integer to string : 1
After converting tuple to dictionary : {'a': 1, 'f': 2, 'g': 3}
```

Simple Input and Output functions:

- input() and print() are widely used for standard input and output operations respectively.
- We use the **print() function to output** data to the standard output device (screen).

```
print('This sentence is output to the screen')
# Output: This sentence is output to the screen

a = 5

print('The value of a is', a)
# Output: The value of a is 5
```

- In the second print() statement, we can notice that a space was added between the string and the value of variable a.This is by default, but we can change it.
- The actual syntax of the print() function is

```
print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)
```

- Here, objects is the value(s) to be printed.
- The sep separator is used between the values. It defaults into a space character.
- After all values are printed, end is printed. It defaults into a new line.
- The file is the object where the values are printed and its default value is sys.stdout(screen). Here are an example to illustrate this.

```
print(1,2,3,4)
# Output: 1 2 3 4

print(1,2,3,4,sep='*')
# Output: 1*2*3*4

print(1,2,3,4,sep='#',end='&')
# Output: 1#2#3#4&
```

- In Python, we have the input() function to allow this. The syntax for input() is

```
input([prompt])
```

- where prompt is the string we wish to display on the screen. It is optional.

```
>>> num = input("Enter a number: ")
Enter a number: 10
>>> num
'10'
```

- Here, we can see that the entered value 10 is a string, not a number. To convert this into a number we can use int() or float() functions.

```
>>> int('10')
10
>>> float('10')
10.0
```